Prerequisite: Definition of necessary functions

```python
# 定义高斯核函数
def k(x:float, mean:float=0, variance:float=1) -> float:
    return 1 / math.sqrt(2 * math.pi * variance) * math.exp(-(x - mean) ** 2 / (2 * variance))
```

✓ 0.0s

```python
# 定义true density function
def f(x:float) -> float:
    return 0.5 * k(x, mean=-2, variance=1) + 0.5 * k(x, mean=2 ,variance=1)
```

✓ 0.0s

```python
# 定义 kernel density estimator
def kde_estimator(x: float, data: list, h: float) -> float:
    n = len(data)
    return sum(k((x - xi) / h) for xi in data) / (n * h)
```

✓ 0.0s

```python
# 定义 L2距离
def l2_error(data: list, h: float, a: float = -10, b: float = 10, num_points: int = 2000) -> float:
    x = np.linspace(a, b, num_points)
    dx = (b - a) / (num_points - 1)

    def kde(x_val):
        return kde_estimator(x_val, data, h)

    integrand = 0.0
    for i, xi in enumerate(x):
        diff = kde(xi) - f(xi)
        val = diff * diff * f(xi)
        if i == 0 or i == num_points - 1:
            integrand += 0.5 * val
        else:
            integrand += val
    integrand *= dx
    return np.sqrt(integrand)
```

✓ 0.0s

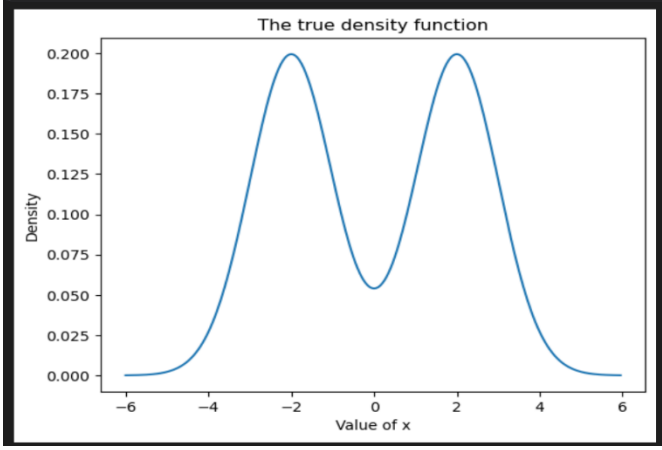
```python
# 定义样本生成函数
def generate_samples(n: int) -> np.array:
    samples = []
    for _ in range(n):
        if np.random.rand() < 0.5:
            samples.append(np.random.normal(-2, 1))
        else:
            samples.append(np.random.normal(2, 1))
    return np.array(samples)
```

✓ 0.0s

Task 1

```python
# Task 1
plt.plot(np.arange(-6, 6, 0.05), [f(x) for x in np.arange(-6, 6, 0.05)])
plt.title("The true density function")
plt.xlabel("Value of x")
plt.ylabel("Density")
plt.show()
```

✓ 0.0s



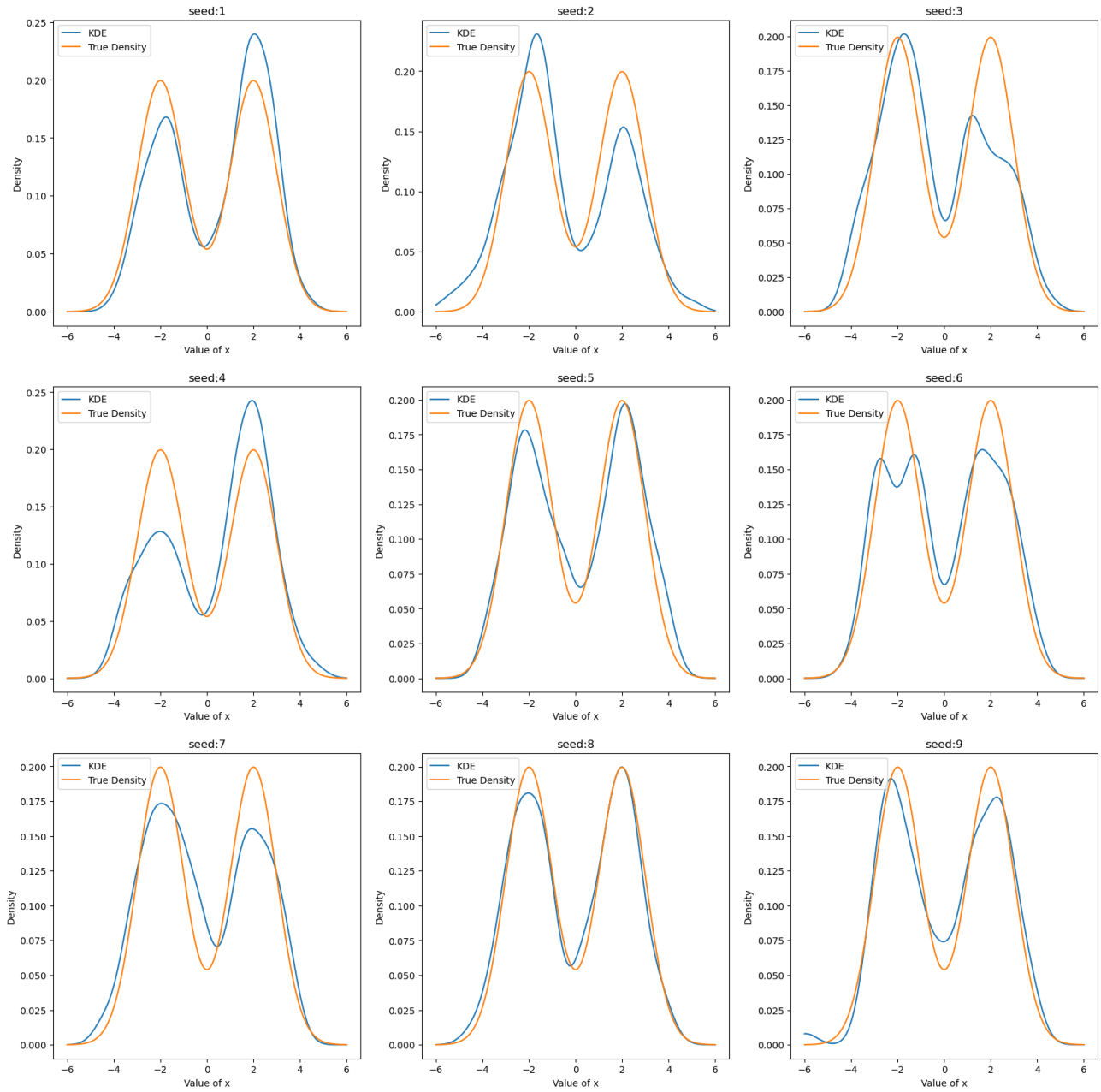
Task 2

```python
# Task 2
fig, ax = plt.subplots(3, 3, figsize=(15, 15))
x_axis = np.linspace(-6, 6, 500)
for seed in range(1, 10):
    # 固定种子
    np.random.seed(seed)

    # 生成数据 然后进一步KDE TRUE DENSITY
    data = generate_samples(100)
    kde = [kde_estimator(x, data, h=0.5) for x in x_axis]
    true_density = [f(x) for x in x_axis]
    row, col = (seed - 1) // 3, (seed - 1) % 3

    # 开画
    ax[row, col].set_title(f"seed:{seed}")
    ax[row, col].plot(x_axis, kde, label="KDE")
    ax[row, col].plot(x_axis, true_density, label="True Density")
    ax[row, col].legend(loc="upper left")
plt.show()
```

✓ 3.8s



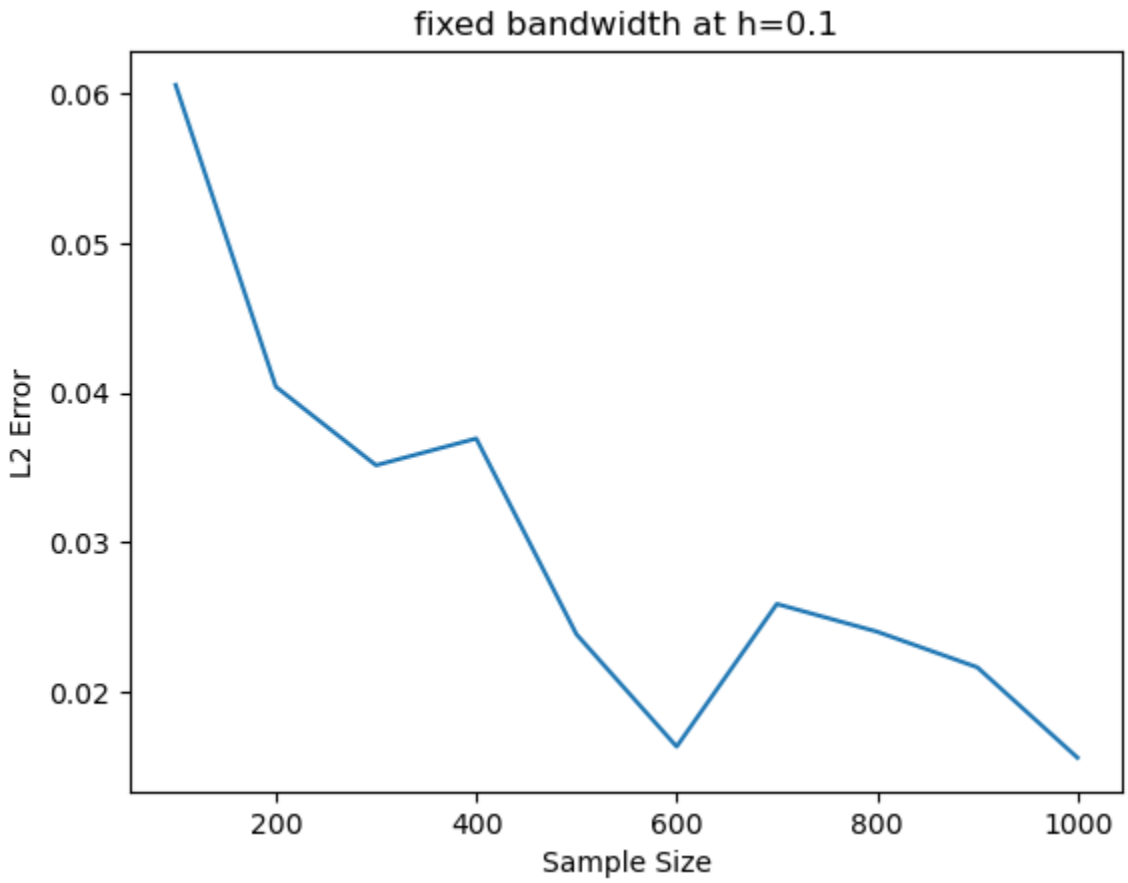
Task 3

In general, increasing the sample size indeed improves estimation under a fixed bandwidth.

```python
# Task 3
np.random.seed(43)
x_axis = []
y_axis = []
for i in range(1, 11):
    # 样本数量
    n = 100 * i
    x_axis.append(n)

    # 计算L2误差项
    data = generate_samples(n)
    l2 = l2_error(data=data, h=0.1)
    y_axis.append(l2)
plt.plot(x_axis, y_axis)
plt.xlabel("Sample Size")
plt.ylabel("L2 Error")
plt.show()
```

✓ 18.8s



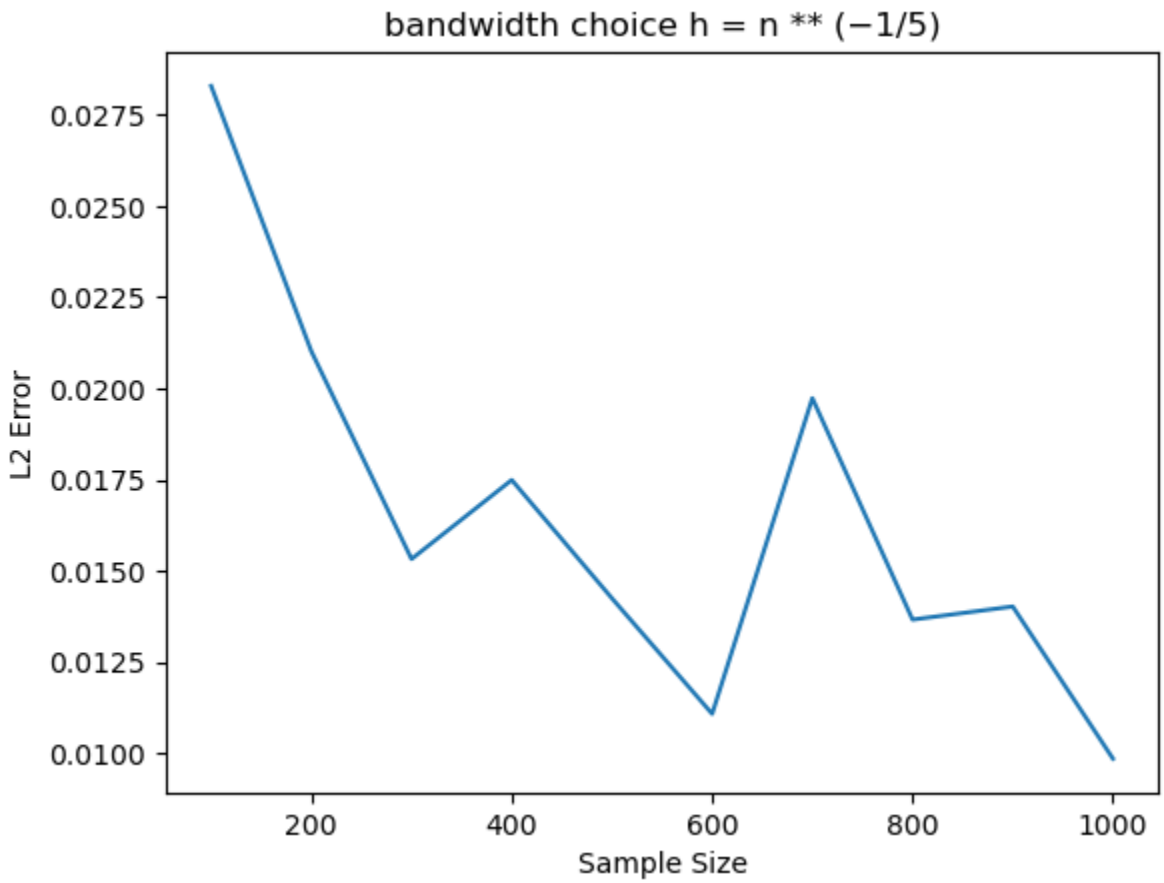
Task 4

This adaptive bandwidth strategy significantly reduces L2 error, greatly improves estimation accuracy compared with the fixed one.

```python
# Task 4
np.random.seed(43)
x_axis = []
y_axis = []
for i in range(1, 11):
    # 样本数量
    n = 100 * i
    x_axis.append(n)

    # 计算L2误差项
    h = n ** (-0.2)
    data = generate_samples(n)
    l2 = l2_error(data=data, h=h)
    y_axis.append(l2)
plt.plot(x_axis, y_axis)
plt.title("bandwidth choice h = n ** (-1/5)")
plt.xlabel("Sample Size")
plt.ylabel("L2 Error")
plt.show()
```

✓ 16.9s



Brief discussion

The first part is about variability across different random samples. In task 2, we fix all other parameters and manipulate random seed only, which results in different random samples. As it is clearly shown in the graph, KDE curves exhibit noticeable variability with the change of random seeds. But it's also worth mentioning that though KDE curves are distinct, all of them preserve the bimodal shape that is consistent with the true density function.

Next, we analyze the effect of sample size on estimation accuracy. In task 3, in an effort to measure the estimation accuracy, we construct the L2 distance function. And just as we did in task 2, we hold all other irrelevant parameters constant, and only change sample size varied from 100 to 1000. The result illustrates that the estimation error exhibits a clear decreasing tendency as the sample size increases. This result comes with no surprise, as it's highly consistent with the Central Limit Theorem.

Lastly, we study the differences between fixed bandwidth and adaptive bandwidth. In task 3, bandwidth is set to be a constant at h=0.1, while in task 4, bandwidth is a formula that depends on sample size n (h=n**(−1/5)). Comparing results from these two, we can find that provided the same sample size, the estimation error generated in task 4 is way smaller than that from the previous one. This comparison illustrates that adaptive bandwidth tends to be more accurate in calculating KDE.